Architecture

How it works and why it's built this way.

For: Everyone

One chapter of the handbook. The full handbook, with every chapter, is handbook.pdf in the same folder.

This is the **langgraph** version. People chat with Granite in Open WebUI. Granite calls a tool, and the tool asks policy-api to run the check. Inside policy-api, each check runs as a LangGraph graph that saves its state to Postgres after every step. Other versions of the product are described in the README.

The job

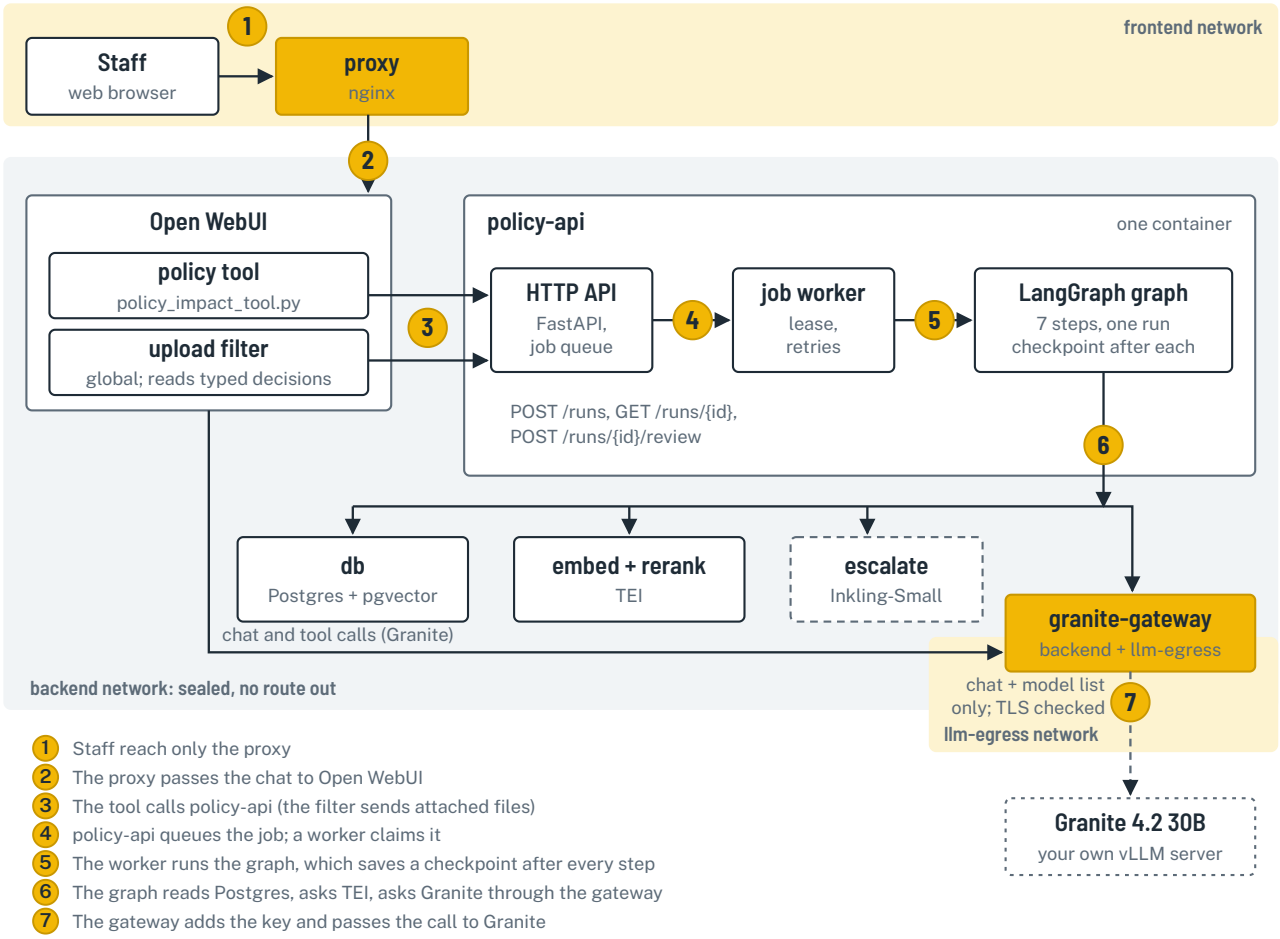
When a head policy changes, find every policy in a chosen group that now conflicts with it, repeats its old wording, points to a section that moved, or uses a defined term whose meaning changed. Missing an affected policy is the expensive mistake, so the design favours **recall** (finding everything) over precision (flagging nothing extra), and it puts a person in the loop for anything uncertain.

The core rule

The chat model never decides impact itself. It works out what the person is asking, calls the tool, and passes on the result. The graph inside policy-api makes every decision. The routing between steps is plain code, not a model's choice. Code builds every number, table and spreadsheet. The chat answer and the Excel file therefore always agree, and every decision can be traced in the audit trail.

With `REVIEW_GATE=1` there is a second rule: the chat model cannot settle a "maybe" either. Only the person's own typed message can (Review pause).

The parts in one picture



The request path. People reach only the proxy; policy-api and Open WebUI reach Granite only through the gateway.

Read it from left to right. People reach only the proxy. Everything that holds policy text sits on the sealed backend network. The only way out is `granite-gateway`, which reaches your Granite server over the `llm-egress` network. The numbers 1 to 7 mark the path of one check; the legend lists them.

Request flow, step by step

A. A question in the chat

Example: "Which Safety policies are affected by the change to POL-007?"

1. The person types in Open WebUI, reached through the `proxy`.
2. Open WebUI sends the chat to Granite through `granite-gateway`, with the system prompt (`openwebui/system_prompt.md`) and the tool list. Function calling is set to **Native**.
3. Granite replies with a tool call, for example `check_policy_change(policy_group, head_policy)`.

4. Open WebUI runs the tool (`openwebui/policy_impact_tool.py`). The tool calls policy-api with `POLICY_API_KEY` . If a group or policy name is not a clear match, the tool returns a question for the person instead of guessing.
5. The tool's result goes back to Granite, which shows it. The tool tells Granite to show answers exactly as written, so numbers, policy IDs and links reach the person unchanged.

The tools are `list_policy_groups` , `find_policy` , `check_policy_change` , `get_check_result` , `make_check_files` , `compare_policies` and `make_document` . There is **no search tool** in this version. A question about what a policy says ("What do we require for lockout devices?") has no tool behind it, and the system prompt tells Granite to say it does not know. The `hermes` branch adds search and policy-text tools.

B. A question with an attached document

1. The person attaches a file. Before the message leaves Open WebUI, the **upload filter** (`openwebui/upload_filter.py` , a global function) sends the file to `POST /uploads` on policy-api with `POLICY_API_KEY` . It reads only files that this person uploaded and that sit in Open WebUI's upload folder.
2. policy-api reads the file in its own limited process, splits it into numbered sections, stores it, and guesses which stored policy it is a new version of.
3. The filter adds a note to the end of the person's message: `[Attached file ...: upload id ...]` . It also stops Open WebUI from cutting the file into its own search index.
4. Granite sees the note and passes the upload id to `check_policy_change` .

C. A policy change check, end to end

1. The tool finds the group (`GET /subsets`) and the head policy (`GET /policies`), then sends `POST /runs` with the asking person's email or name as `requested_by` . policy-api stores a `runs` row and a row in the `jobs` table, and answers **202 Accepted** with a run id.
2. A worker inside policy-api claims the job (`FOR UPDATE SKIP LOCKED`) with a 60-second lease and renews the lease every 20 seconds while it works. The run's status becomes `running` .
3. The worker starts the graph with the run id as the **thread id**. The graph runs seven kinds of step (The graph). After each step, LangGraph writes a checkpoint to Postgres and waits for the write to finish before the next step starts (`durability="sync"`).
4. While this runs, the tool asks `GET /runs/{id}` every 3 seconds (`POLL_SECONDS`) and shows the `progress` text in the chat. The tool gives up waiting after `MAX_WAIT_MINUTES` (30), but the check carries on.
5. If the graph finishes, the worker records `done` and the result. `GET /runs/{id}` then returns the chat answer (built in code) and a signed Excel link that works for 72 hours.
6. If the graph reaches the review step with `REVIEW_GATE=1` , the check pauses. See Review pause.
7. If the person then asks for a PDF, Word file or charts, Granite calls `make_check_files` .

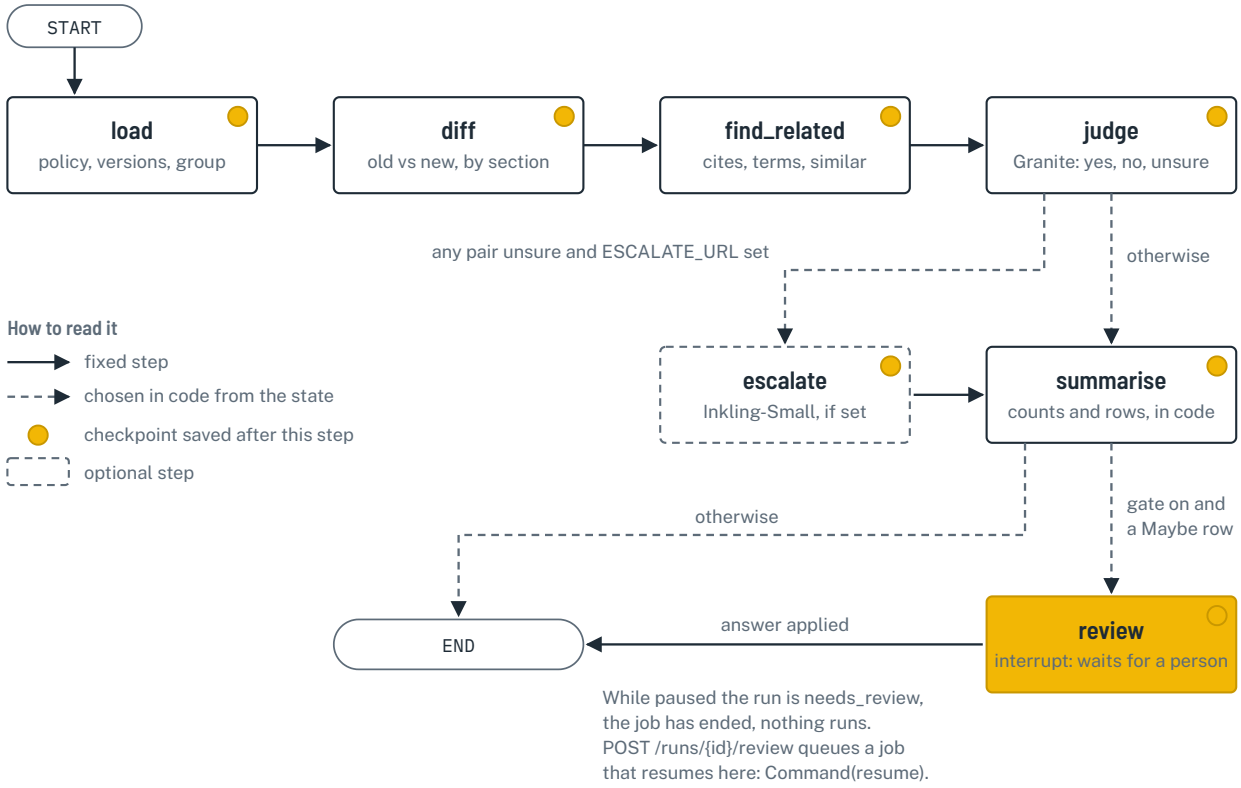
Retries. A step that fails for a temporary reason (database blip, model restart) raises an error. The worker waits (random, growing up to 5 minutes) and tries the job again, up to `JOB_MAX_ATTEMPTS` (3). The retry **carries on from the last saved checkpoint**; it does not start over. A problem with the question itself (no such policy, nothing changed) is a `RunError` : it is shown to the person at once and is not retried.

Crash and resume. If policy-api is stopped or dies mid-check, the job's lease runs out (or, after a restart, the worker takes its own jobs back at once). Any worker claims the job again. The worker asks LangGraph for the thread's saved state. If steps are left, it continues from the first unfinished one. The people waiting in Open WebUI see "Starting again after an interruption…". Work inside one step is repeated: for example, a half-finished `judge` step asks its pairs again. The step first deletes the verdicts it had already saved for its stage, so the audit trail never holds duplicates.

D. Deciding "maybe" sections (`REVIEW_GATE=1`)

1. The graph reaches `review`. It calls `interrupt()`, which saves a checkpoint and stops. The worker sets the run to `needs_review` and finishes the job. Nothing is running or queued.
2. The tool sees `needs_review`, shows the list of "maybe" sections and tells the person what to type (see the message format below).
3. The person types a message that starts with `decide`: `decide 3f9a1c2e SAF-150 3.1 yes, SAF-151 2.4 no`. The 8 characters are the **check number**. `decide 3f9a1c2e all no` answers every listed section at once.
4. The upload filter reads this from the person's own message only, never from an assistant or tool message. It checks the format and sends `POST /runs/{id}/review` with the person's Open WebUI name as `reviewed_by`.
5. policy-api checks that each section is on the waiting list, then **claims** the check with one atomic update (`needs_review` to `queued`). Two answers sent together cannot both apply. It queues a `review_run` job.
6. A worker takes that job and resumes the graph with `Command(resume=answer)`. The `review` step turns each "yes" into a confirmed "Yes" row and drops each "no". Sections the person left out stay "Maybe". The check ends as `done`.
7. The filter adds a note to the person's message ("decisions were recorded"). Granite then calls `get_check_result` to show the final answer.

The graph



One check as a graph. Code chooses every branch; the state is saved to Postgres after every step.

The graph is in `policy_api/app/graph.py`. It is built with LangGraph's `StateGraph`. The two model steps ask about one pair at a time; every other step is ordinary code.

Solid arrows are fixed steps. The two labelled branches are chosen by small functions in code, from the state alone. After every step, the whole state is saved to Postgres.

The state

The state (`RunState`) holds plain JSON values only, so a saved checkpoint never depends on our class layout. Each step returns only the keys it changes.

Key	Holds	Set by
`run_id`	The check's id. Also the LangGraph thread id	Input
`head`	Head policy id and title	`load`
`labels`	Names of the two versions compared ("version 3", "uploaded file ...")	`load`
`policy_ids`, `metas`	Policies in the group (without the head policy) and their titles and owners	`load`

Key	Holds	Set by
`changes`	Each changed section, with its saved `changes` row id	`diff`
`pairs`	Each (changed section, related section) pair: the row, how it was found, the verdict, and whether Inkling-Small saw it	`find_related`, then `judge`, then `escalate`
`errors`	How many pairs the model could not answer	`judge`
`result`	The finished answer: counts, rows, not-affected list, method. Chat text and Excel both come from it	`summarise`, then `review`
`reviewed`	The person's decisions	`review`

The steps

Step	What it does	Reads	Writes
`load`	Finds the head policy and its two versions (or the stored version and the upload). Lists the group's other policies. Fails with a plain message if something is missing	`runs`, `policies`, `uploads`	`head`, `labels`, `policy_ids`, `metas`
`diff`	Clears any earlier attempt's rows, compares old and new text section by section (in a thread, with a time cap), saves each change	`policy_versions`, `uploads`	`changes`; `changes` table
`find_related`	For each change, collects sections in the group that cite the head policy, use a changed defined term (both always kept), or are similar in meaning (embedding search, then reranker keeps the top N)	`changes`, `refs`, `clauses`, TEI	`pairs`
`judge`	Granite reads each pair and answers yes, no or unsure, with a reason, quotes and a suggested edit, forced into a JSON schema. Up to `JUDGE_CONCURRENCY` (16) at once. Each verdict is saved. A pair the model cannot answer becomes "unsure" with a note	`pairs`	`pairs`, `errors`; `judgements` table
`escalate`	Optional. Sends only the "unsure" pairs to Inkling-Small for a second opinion. If it fails, the first opinion stays	`pairs`	`pairs`; `judgements` table
`summarise`	Per policy: Yes if any section is "yes", Maybe if any is "unsure", otherwise not affected. Pure code	all of the above	`result`
`review`	Only with `REVIEW_GATE=1` and at least one Maybe row. Pauses until a person answers, then applies the answer	`result`	`result`, `reviewed`

The branches

After	Goes to	When
`judge`	`escalate`	Any pair is "unsure" **and** `ESCALATE_URL` is set
`judge`	`summarise`	Otherwise
`summarise`	`review`	`REVIEW_GATE` is on **and** the result has at least one Maybe row
`summarise`	end	Otherwise

The other connections are fixed: `load` to `diff` to `find_related` to `judge`, `escalate` to `summarise`, and `review` to end. The path through the graph is a straight line with two optional detours. It has no loops, so it cannot run on forever.

Recursion limit. Every run is started with `recursion_limit=25`. The graph has at most seven steps per run. A run that came near 25 would be a bug, and LangGraph would stop it with an error rather than let it spin.

In `find_related`, the sections that cite the head policy or use a changed term are never cut by the reranker, because explicit links are where most real impact is. The similar-meaning search catches policies that overlap in meaning without citing the head policy.

Checkpoints

	Production	Demo mode and most tests
Checkpointer	`AsyncPostgresSaver` (`langgraph-checkpoint-postgres` 3.2.0), using `DATABASE_URL`	`InMemorySaver`
Where saved	Tables in the same Postgres database as the policies	In the policy-api process
Survives a restart	Yes: a check resumes, and a paused check stays paused	No: a restart loses the state
Tables	Created on every start by `saver.setup()` (safe to repeat)	None

`open_checkpointer` picks Postgres whenever `DEMO_MODE` is off and `DATABASE_URL` is set. The tests that use `TEST_DATABASE_URL` use Postgres too (testing.md).

What is stored where

Store	Tables	Holds	Who writes it
Checkpoints	`checkpoints`, `checkpoint_blobs`, `checkpoint_writes`, `checkpoint_migrations`	The graph's state after each step, per thread (one thread per check). This is how a check resumes	LangGraph
Application	`runs`, `changes`, `judgements`, `uploads`	The check's status, progress, final result; each change; every model verdict with its exact prompt (the audit trail); uploaded files	policy-api
Job queue	`jobs`	`impact_run` and `review_run` jobs with lease, attempts and last error	policy-api
Policies	`policies`, `policy_versions`, `clauses`, `refs`, `subsets`	The policy library and its reference graph	`app.ingest`

The checkpoint tables are for **resuming**. The application tables are for **reading and auditing**. The status the chat shows always comes from `runs`, not from the checkpoints. Checkpoints are not cleaned up automatically; they are small, and keeping them means a finished check's steps can be inspected. To purge them with old runs, see security.md.

Review pause (`REVIEW_GATE`)

`REVIEW_GATE` is off by default. When it is off, "maybe" results go into the answer and the Excel file, and people decide them afterwards (the Review decision column, or the review queue). When it is on, a check that has "maybe" rows stops at `review` and waits.

What the API shows.

Route	Use	Notes
`GET /runs/{id}`	Status, progress and error. While `status` is `needs_review` it also returns `review`, a list of `policy_id`, `policy_title`, `clause`, `clause_heading` and `why` for each waiting section. When `done`, the answer and Excel link	Accepts the full id **or** the 8-character check number
`POST /runs/{id}/review`	Body: `decisions` (a list of `policy_id`, `clause`, `affected`) and optional `reviewed_by`. Answers 202 and `queued`	Accepts the full id or the 8-character check number. For a short number, only paused checks are searched

Every other route needs the full id. The answers to a bad review request are plain: 404 for no such check, 409 if the check is not waiting (or was just answered by someone else, or two checks match a short number), 422 if a section is not on the waiting list.

The check number is the first 8 hex characters of the run id, for example `3f9a1c2e`. It is only 32 bits and is not tied to the person who asked (Limits).

Partial answers. The person can decide some sections and leave the rest. The check then finishes with the undecided ones still marked Maybe. There is no second pause.

Never answered. A paused check waits for ever. No timer closes it. It uses no worker slot and no model. Its state is safe in Postgres, so it survives restarts. Ask the assistant for the check number and the list is shown again (operations.md).

Components

Component	Image and version	Role	Networks
`proxy`	`nginx:1.27-alpine` (`NGINX_TAG`)	The only thing people reach: Open WebUI, plus report downloads and `/health`. Holds no data	backend + frontend
`open-webui`	`ghcr.io/open-webui/open-webui` (`OPENWEBUI_TAG`, pinned in `.env`)	Chat screen. Sends chats to Granite. Runs the tool and the upload filter. Trimmed: no web search, memories or code running	backend
`policy-api`	Built from `policy_api/` (Python 3.12, FastAPI 0.115 or later, LangGraph 1.2.16, `APP_VERSION`)	The HTTP API, the job queue and worker, the graph, uploads, reports and downloads	backend
`db`	`pgvector/pgvector:pg17` (`PGVECTOR_TAG`)	Postgres with pgvector: policies, clauses, graph, runs, jobs, audit trail, and the checkpoint tables	backend

Component	Image and version	Role	Networks
`embed` , `rerank`	`text-embeddings-inference` (`TEI_TAG`) with Granite embedding and reranker models	Embeddings for similarity search; reranker to order and cut candidates	backend
`granite-gateway`	`nginx:1.27-alpine`	The backend's only way out. Passes chat and model-list calls to `GRANITE_URL` , adds the Granite key, checks TLS	backend + llm-egress
Granite 4.2 30B	Your own vLLM server, outside this stack (vLLM 0.20 or later)	The chat model, and the judge for every pair	Reached only through the gateway
`escalate` (optional)	`vllm/vllm-openai` (`VLLM_TAG`), Inkling-Small	Second opinion on "unsure" pairs	backend
`extract` (optional)	`vllm/vllm-openai` (`VLLM_TAG`), gpt-oss-20b	Finds references the regex misses during ingest	backend
Review and training services (optional)	Argilla v2.8.0 and its helpers; `lora-trainer` ; `judge-candidate` ; `judge-tuned`	Review queue and fine-tuning (review-and-training.md)	backend

The graph runs **inside** `policy-api` . There is no separate graph or agent container. The tool and the upload filter run inside Open WebUI, from files pasted into its admin screens (setup.md).

Network and security boundaries

```
frontend network:   people → proxy
backend network:    proxy → open-webui → policy-api → db, embed, rerank
backend + egress:   open-webui and policy-api → granite-gateway → Granite server
```

- `backend` is `internal: true` : no route out. `proxy` is the only container on `frontend` . The gateway is the only container on `llm-egress` .
- The gateway passes `/v1/chat/completions` and `/v1/models` and nothing else, and checks the Granite server's certificate.
- policy-api answers only requests with the right key and a known `Host` name. The proxy shows people only Open WebUI, report downloads and `/health` .
- The checkpoints are in the same database as the policies and add no new network path.

Four keys keep the parts apart. Each is held by as few parts as possible.

Key	Where it lives	Who uses it	What it protects
`POLICY_API_KEY`	`.env`	policy-api (to check it); the tool and the upload filter (a setting, called a valve, in Open WebUI)	Every policy-api route except signed downloads
`GRANITE_API_KEY`	`.env`	`granite-gateway` only	The Granite server. Open WebUI and policy-api never see it
`LINK_SIGNING_KEY`	`.env`	policy-api	Download links

Key	Where it lives	Who uses it	What it protects
`LLM_API_KEY`	`.env`	policy-api and the optional local models	The optional `escalate`, `extract` and trained-judge servers

See security.md.

When things fail

What fails	What happens
Granite server down or slow	Chat: Open WebUI shows an error. Check: each pair that fails is marked "could not be checked automatically" and shows as Maybe. If **every** pair fails, the check stops with "The checking model is not responding" and is not retried. Run it again later
Worker or policy-api crashes mid-graph	The lease runs out (or the restarted worker takes its job back). The job runs again and the graph resumes after the last saved step. The person sees "Starting again after an interruption…"
A temporary error inside a step (database blip, TEI restart)	The job is retried up to `JOB_MAX_ATTEMPTS` (3) with growing waits, each time from the last checkpoint
The job fails every time	After the last attempt the job is `dead` and the run is `failed`. The person sees "failed several times in a row". The error stays in `jobs.last_error` (operations.md)
A problem with the question (no such policy, nothing changed)	Not retried. The person gets a plain reason
Review never answered	The check stays `needs_review` indefinitely. No timeout, no worker used. Anyone with the check number can still answer it later
Two answers to the same review at once	The first claims the check. The second gets "not waiting for a decision" (409)
policy-api down	The tool says the service did not respond. A queued check waits in the database
Postgres down	Everything stops, including checkpointing. Work resumes when it is back
Checkpoint tables lost (restored from an older backup)	A paused or half-finished check cannot resume. Start it again. `runs` and `judgements` are unaffected
A check takes longer than 30 minutes of waiting	The tool stops waiting and tells Granite the check is still running. The check itself continues. The person asks again with the check number
A file cannot be read	The upload filter puts a plain note in the message; Granite tells the person

Limits

- **Check numbers are not tied to the requester.** Anyone who has seen a check number can decide that check, from any chat. The decision is recorded under the Open WebUI name of whoever typed it, so it can be traced. See security.md.
- **Everyone with an account can ask about every group.** There is no per-user group filter.

- **Training still targets gpt-oss-20b.** The fine-tuning scripts and the `judge-candidate` and `judge-tuned` services are set up for that model, not for Granite. Verdicts can be sent to a tuned judge; the chat stays on Granite (review-and-training.md).
- **Decisions in chat are not used for training.** They change the final answer. They are not copied to `judgements`. The review queue is the source of training data.
- **No search or policy-text tool.** This version answers only about checks, comparisons and files.
- **The Granite server must be set up for this.** Tool calling, structured output and a context of at least 32768 tokens (setup.md).
- **A very long check holds the chat open.** The tool waits up to 30 minutes (`MAX_WAIT_MINUTES`). Open WebUI, or a proxy you run in front of the stack, may give up sooner. The check carries on either way, and the result is fetched later by check number.
- **The chat model follows its rules by instruction, not by force.** It can word an answer badly. That is why numbers, links and verdicts are produced by code and passed through unchanged, and why only the filter, never the model, can record a decision.

Why these choices

Choice	Why	Instead of
A graph with fixed routing, run by code	The steps never change. Routing is deterministic, testable and auditable	An agent that plans its own steps (more tokens, results vary between runs, harder to audit)
LangGraph with Postgres checkpoints	A crash resumes after the last finished step instead of repeating a long, costly run. Pausing for a person is built in (`interrupt`)	A hand-written state table and pause logic
The graph inside policy-api	No new container to run, secure or back up. Checkpoints live in the database that is already backed up	A separate workflow server (Temporal, Dagster): another stateful system inside the contained network
Review pause as a graph interrupt	The paused check holds no process and no model. It survives restarts	Keeping a worker waiting, or a custom "waiting" status with its own resume code
Decisions typed by the person, read by the filter	A model cannot approve its own "maybe". Prompt injection in a policy cannot record a decision	A tool the model calls to record a decision
Granite 4.2 30B as judge and chat model	US-developed (IBM), Apache 2.0, dense 30B that fits one 80 GB GPU in bf16, 128K context, tool calling, thinking can be switched off per request for fast, repeatable verdicts	Inkling for everything (needs 4–8 high-end GPUs per replica)
Use the Granite server you already run, behind one gateway	No second copy of a 58 GB model to host and update. One narrow, checked way out (two routes, one address, key held in one place) instead of a model container inside the stack	Starting a local judge container, or letting each service call the server on its own
Inkling-Small only for "unsure"	Larger-model judgement where it matters, at a fraction of the cost	Running every pair through the large model

Choice	Why	Instead of
Reranker before judging	Cuts hundreds of similar clauses to about 20 per change; the biggest compute saving	Judging every similar clause
JSON-schema guided decoding	Every verdict is valid and machine-readable	Asking for JSON in the prompt and hoping
Postgres for the graph of references	Recursive queries cover policy hierarchies; one fewer system to run	A separate graph database
Excel built with openpyxl	Exact, repeatable, safe from formula injection	Letting the model write the spreadsheet
Job queue in Postgres (`FOR UPDATE SKIP LOCKED`, leases)	Runs survive restarts with no new service to run or secure; Postgres is already here	Temporal or Dagster, for a queue of a few hundred jobs a day

Data model (db/init.sql)

- `policies`, `policy_versions`, `clauses`: every version is kept. `current_clauses` is a view of each policy's latest version.
- `refs`: which clause cites which policy and section. `subsets`: which policies are in which group.
- `uploads`, `runs`, `changes`: each question and what changed.
- `judgements`: one row per model decision, with model name, the exact prompt and its hash, verdict, and the reviewer's decision. This is the audit trail and the training data.
- `jobs`: the work queue (see operations.md). `schema_migrations` records which files in `policy_api/app/migrations/` have been applied; `db/init.sql` is the starting point for a new database.
- `checkpoint_*` tables: made by LangGraph on start, not by `db/init.sql`.

Clause parsing rules

A new clause starts at a line that begins with a section number:

- a Markdown heading with a number (`## 4 Lockout devices`)
- any multi-level number (`4.2 Devices are inspected…`, `4.2.1) …`)
- a single number followed by a short title (`4 Lockout devices`)

`1. Do this.` inside a clause stays part of that clause. Text before the first number is clause `0`. Policy IDs are found with `POLICY_ID_PATTERN` (default `[A-Z]{2,6}-\d{1,5}`), in forms like `POL-007 §4.2`, `§4.2 of POL-007`, `POL-007 section 4` and `POL-007` alone (whole policy).

Roadmap

Built since v0.1: the job queue, the review queue, LoRA fine-tuning, reports and charts, and in this version the LangGraph run with checkpoints and the review pause.

Next: OCR for scanned PDFs and Word's automatic numbering (known limits), and Kubernetes manifests for models that need more than one node.